Lập trình mạng · Chương 4

gRPC: streaming, deadline và mã trạng thái

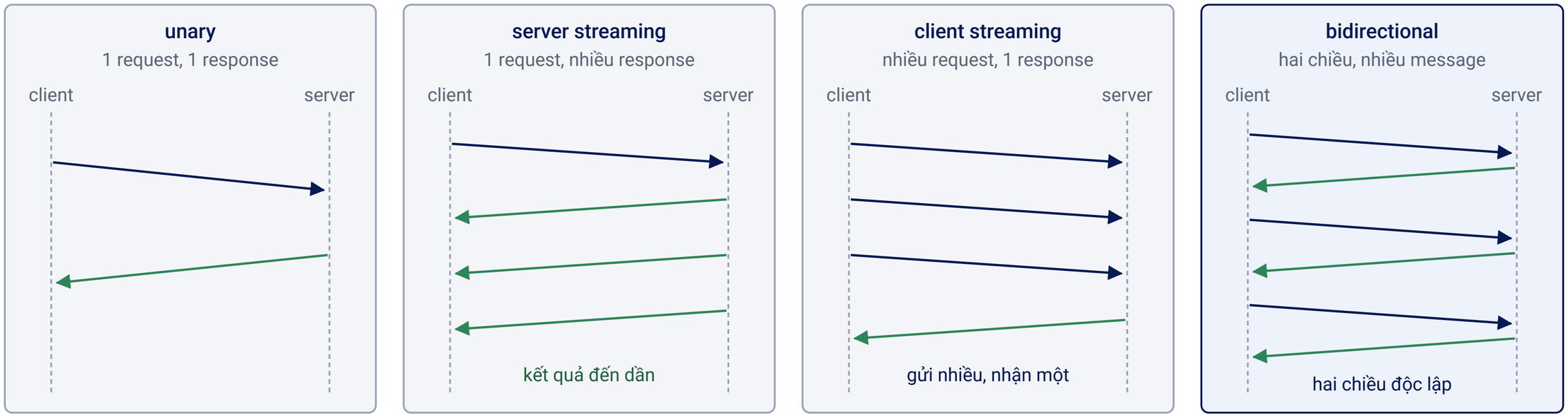
TS. Đặng Ngọc Hùng

Nhiều kết quả, giới hạn thời gian và báo lỗi: ba nhu cầu mới

1. Liệt kê địa danh trong một vùng: hàng nghìn kết quả; với unary, client chờ cho tới khi có **một** message lớn
2. Lời gọi đi qua nhiều dịch vụ: cần **giới hạn thời gian** chung cho cả chuỗi
3. Lời gọi thất bại: cần cách báo **thống nhất** cho client ở mọi ngôn ngữ

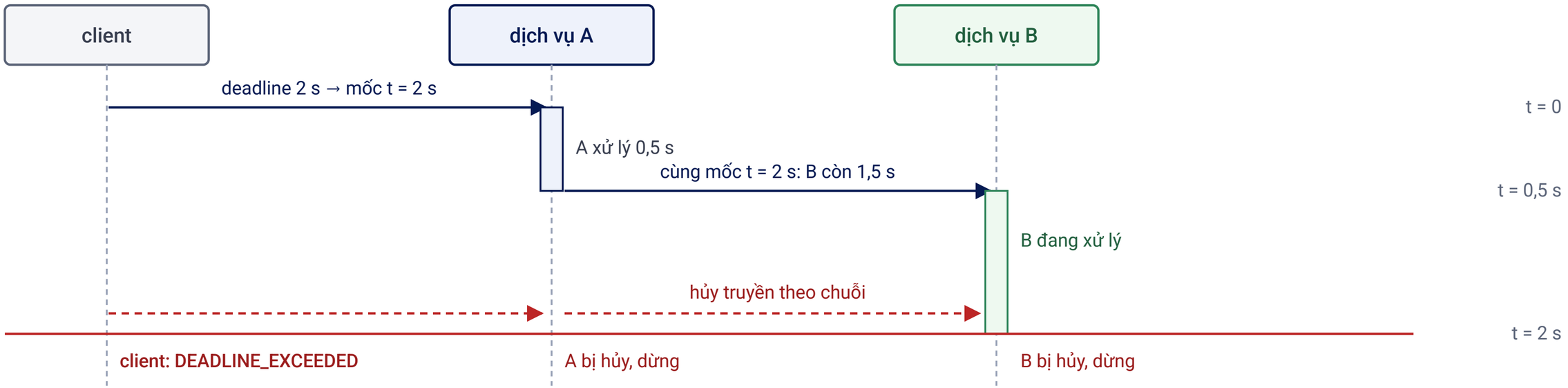
> **Câu hỏi của phần:** gRPC trả nhiều message trên một lời gọi thế nào, đặt deadline ra sao, và báo lỗi bằng gì?

Bốn kiểu RPC: từ khóa stream đánh dấu phía gửi nhiều message



Bốn kiểu RPC: unary, **server streaming**, **client streaming**, **bidirectional streaming**; mỗi lời gọi là một HTTP/2 stream.

Deadline là một mốc thời gian, truyền theo chuỗi lời gọi



Deadline là một mốc thời gian; gRPC mang mốc đó theo các lời gọi tiếp theo và hủy cả chuỗi khi hết giờ (**DEADLINE_EXCEEDED**).

Bốn RPC trong một dịch vụ; stream phía client qua async stub

Protobuf

```
service RouteGuide {
  rpc GetFeature(Point) returns (Feature) {}                    // unary; tọa độ sai: mã INVALID_ARGUMENT
  rpc ListFeatures(Rectangle) returns (stream Feature) {}       // server streaming: trả địa danh dần
  rpc RecordRoute(stream Point) returns (RouteSummary) {}       // client streaming: gửi lộ trình, nhận tổng kết
  rpc RouteChat(stream RouteNote) returns (stream RouteNote) {} // bidirectional streaming
}
```

Kết quả chạy

```
RecordRoute (client streaming):
+  10 ms  sent 21.0288, 105.8360
+ 422 ms  sent 21.0368, 105.8347
+ 825 ms  sent 21.0479, 105.8368
+1244 ms  sent 21.0431, 105.8580
+1679 ms  summary: 4 points, 4 features, 4415 m, 1 s
RouteChat (bidirectional streaming):
+  19 ms  sent "tin 1" at 21.0288, 105.8525
+ 328 ms  sent "tin 2" at 21.0288, 105.8525
+ 397 ms  received "tin 1" at 21.0288, 105.8525
+ 650 ms  sent "tin 3" at 21.0288, 105.8360
+ 961 ms  sent "tin 4" at 21.0288, 105.8525
+ 977 ms  received "tin 1" at 21.0288, 105.8525
+ 980 ms  received "tin 2" at 21.0288, 105.8525
+1274 ms  server completed
```

Server streaming: gửi từng địa danh ngay khi tìm được, client nhận dần

Java

```
for (Feature feature : Features.ALL) {
    if (!inRange(feature.getLocation(), request)) {
        continue;
    }
    sleep(300);  // mô phỏng mỗi địa danh tốn 300 ms để tìm
    call.onNext(feature);  // gửi ngay địa danh này
    sent++;
}
call.onCompleted();
```

Kết quả chạy

```
+ 339 ms  Hồ Hoàn Kiếm
+ 640 ms  Văn Miếu - Quốc Tử Giám
+ 949 ms  Nhà hát Lớn Hà Nội
+1276 ms  Lăng Chủ tịch Hồ Chí Minh
+1584 ms  Chùa Trấn Quốc
+1899 ms  Cầu Long Biên
+1900 ms  completed
```

Hết deadline: client nhận DEADLINE_EXCEEDED, server biết để dừng

Java

```
stub = stub.withDeadlineAfter(Long.parseLong(args[0]), TimeUnit.MILLISECONDS);  // deadline cho lời gọi
Iterator<Feature> features = stub.listFeatures(center);
} catch (StatusRuntimeException e) {
    System.out.printf("+%4d ms  %s%n", (System.nanoTime() - start) / 1_000_000, e.getStatus().getCode());
```

Kết quả chạy

```
+ 336 ms  Hồ Hoàn Kiếm
+ 644 ms  Văn Miếu - Quốc Tử Giám
+ 953 ms  Nhà hát Lớn Hà Nội
+1020 ms  DEADLINE_EXCEEDED
```

- Server kiểm tra `isCancelled()` trước mỗi lần gửi: log ghi `cancelled by client after 3 features`

Mã trạng thái báo lỗi thống nhất; luôn đặt deadline

```java
RouteGuideGrpc.newBlockingStub(channel)
        .withDeadlineAfter(2, TimeUnit.SECONDS)  // luôn đặt deadline: gRPC mặc định không có
        .getFeature(point);
} catch (StatusRuntimeException e) {
```

Kết quả chạy

```
port 9199, point 105.7871, 20.9809: INVALID_ARGUMENT (latitude 105.7871 out of [-90, 90])
port 9198, point 20.9809, 105.7871: UNAVAILABLE (io exception)
```

- **Mã trạng thái gRPC**: INVALID_ARGUMENT do server chọn, UNAVAILABLE do thư viện báo; client đọc bằng `getStatus()`
- gRPC **mặc định không có deadline**: server treo thì lời gọi chờ không giới hạn

Câu hỏi 1

Liệt kê địa danh trong một vùng có thể trả 10 000 kết quả. Nên khai báo RPC này theo kiểu nào?

A. Unary, trả một message chứa cả 10 000 địa danh.

B. Client streaming, để client gửi từng địa danh.

C. Bidirectional streaming, vì dữ liệu lớn thì bắt buộc dùng hai chiều.

D. Server streaming: server gửi từng địa danh ngay khi tìm được.

Câu hỏi 1

Liệt kê địa danh trong một vùng có thể trả 10 000 kết quả. Nên khai báo RPC này theo kiểu nào?

A. Unary, trả một message chứa cả 10 000 địa danh.

B. Client streaming, để client gửi từng địa danh.

C. Bidirectional streaming, vì dữ liệu lớn thì bắt buộc dùng hai chiều.

D. Server streaming: server gửi từng địa danh ngay khi tìm được.

Một request, nhiều kết quả: server streaming cho client nhận dần từng địa danh. Unary buộc server tìm đủ rồi gửi một message lớn.

Câu hỏi 2

Client hết deadline nên lời gọi bị hủy. Phía server có biết để dừng xử lý không?

A. Có: việc hủy được báo tới server; mã server kiểm tra isCancelled hoặc đăng ký lắng nghe việc hủy để dừng.

B. Không: server luôn chạy hết rồi mới biết.

C. Chỉ biết nếu client gửi thêm một request báo hủy.

D. Mã của server bị dừng ngay lập tức, không cần kiểm tra gì.

Câu hỏi 2

Client hết deadline nên lời gọi bị hủy. Phía server có biết để dừng xử lý không?

A. Có: việc hủy được báo tới server; mã server kiểm tra isCancelled hoặc đăng ký lắng nghe việc hủy để dừng.

B. Không: server luôn chạy hết rồi mới biết.

C. Chỉ biết nếu client gửi thêm một request báo hủy.

D. Mã của server bị dừng ngay lập tức, không cần kiểm tra gì.

gRPC báo việc hủy cho server (ServerCallStreamObserver.isCancelled, Context của lời gọi), nhưng không dừng mã của server giữa chừng: mã phải tự kiểm tra.

Tổng kết

1. **Server streaming, client streaming, bidirectional streaming**: chuỗi message qua **StreamObserver**

2. Deadline là một mốc thời gian, truyền theo chuỗi lời gọi; hết giờ: **DEADLINE_EXCEEDED**, server dừng khi bị hủy

3. **Mã trạng thái gRPC** báo lỗi thống nhất; gRPC mặc định không có deadline

Từ khóa cần nhớ server streaming · client streaming · bidirectional streaming · StreamObserver · mã trạng thái gRPC · DEADLINE_EXCEEDED

Mở rộng

Chủ đề nâng cao

- Interceptor của gRPC (ghi log, xác thực)
- Cân bằng tải phía client

Bài tập mở rộng

Đặt deadline cho RecordRoute ngắn hơn thời gian gửi cả lộ trình; quan sát mã trạng thái phía client và onError phía server.

Tiếp theo

Message broker: RabbitMQ, exchange, queue và binding